

Counter patterns

Counters are small sequential machines that change state on each clock edge

Count, wrap, or hold

- A four-bit up counter naturally wraps from fifteen back to zero because the register width
- A down counter subtracts one and wraps under zero back to fifteen
- A modulo-ten counter checks for nine, then reloads zero on the next edge
- Count enable is the preferred hold pattern
- Gray counting is different from plain binary because adjacent codes flip one bit

Browser lab

Digital Design and Verification Platform

HomeTools

Home / Tools / Counter lab

INTERACTIVE TOOL

Counter lab

Step a 4-bit counter through **up**, **down**, **modulo**, **enable**, and **Gray** modes — wrap, hold, and one-bit-change sequences.

Starter example: up counter at 0 — step the clock to increment; try enable hold and Gray one-bit flips.

Load starter example

Challenges (1/22)

Quiz: **up**. Counter that adds 1 each clock is an? Answer: up

Answer

Show hint

Check

Next

Idle

quiz-up

quiz-down

quiz-modulo

quiz-gray

starter

preset-up

preset-down

preset-modulo

preset-enable

preset-gray

step

inc

wrap

hold

toggle-ce

mod-wrap

gray-one

reset

demo

explain

code-enable

full-path

Core ideas

Up

Add 1 each edge; wrap at 2^N .

Down

Subtract 1; wrap under zero.

Modulo

Period $\neq$ power of two (e.g. 10).

Enable

if (ce) count — hold otherwise.

Gray

Adjacent codes differ by one bit.

Counter

Status

Counter lab starter

learn_verilog — counter lab

Real Verilog practice

```
wsl - module14-counter-lab/examples/navigation

# Track A - real Unix
# real Verilog session (Track A)

$ cat examples/counter_demo/counter_demo.v
// counter_demo.v - up and modulo counters
module up_counter4(
    input    clk,
    input    rst,
    input    ce,
    output reg [3:0] q
);
    always @(posedge clk) begin
        if (rst) q ≤ 4'd0;
        else if (ce) q ≤ q + 4'd1;
    end
endmodule

module mod10_counter(
    input    clk,
    input    rst,
    output reg [3:0] q
);
    always @(posedge clk) begin
        if (rst) q ≤ 4'd0;
        else if (q == 4'd9) q ≤ 4'd0;
        else q ≤ q + 4'd1;
    end
endmodule

$ iverilog -t null examples/counter_demo/counter_demo.v
(syntax check passed)
```

Real Verilog - up and modulo counters

Pitfalls to watch

- Do not use blocking assignment in clocked counter RTL
- Do not forget the explicit wrap condition for modulo counters that are not powers of two
- Do not implement hold by assigning from multiple drivers
- Do not confuse Gray count outputs with ordinary binary values

Your turn

- Complete the checklist for at least one track, preferably both
- In the browser, step the modulo mode until it wraps and explain why it returns to zero
- On real Verilog, write an enabled counter that holds when CE is low
- When you are ready